Національний університет «Одеська Політехніка»

Інститут комп’ютерних систем

Кафедра інформаційних систем

Лабораторна робота №7

з дисципліни «Забезпечення якості та контроль якості і тестування ПП»

Тема: «Continuous Integration у процесах тестування»

Виконав:

студент групи АІ-243

Гаврилов О.В.

Перевірили:

Баськов І.О.

Одеса 2026

Мета: Ознайомитися з принципами Continuous Integration (CI) та навчитися налаштовувати CI-процеси для автоматичного запуску тестів у GitHub.

Завдання:

1) Налаштувати Continuous Integration (CI) за допомогою конфігураційного файлу GitHub Actions у репозиторії з існуючими автоматичними тестами та перевірити успішне виконання CI-процесу при відправленні коду в головну гілку main.

2) Перевірити роботу CI-системи за негативним сценарієм: створити нову гілку testing-ci-process із навмисно зламаним кодом, сформувати Pull Request та переконатися, що система виявила помилку і заблокувала злиття.

Хід виконання роботи:

1) Для виконання роботи обрано репозиторій з налаштованими E2E-тестами (Playwright) із попередньої лабораторної роботи;

2) У кореневій директорії проєкту створено структуру папок .github/workflows/ та додано конфігураційний файл ci.yml із налаштуваннями GitHub Actions;

3) Відправлено (push) конфігурацію до головної гілки main та у вкладці Actions перевірено успішний автоматичний запуск і проходження всіх тестів;

4) Створено нову тестову гілку testing-ci-process, в яку навмисно внесено зміни, що ламають логіку роботи застосунку;

5) Створено Pull Request із гілки testing-ci-process до main, після чого автоматична система CI запустила тести, успішно виявила помилку та заблокувала злиття неробочого коду.

Завдання №1

Для налаштування Continuous Integration у кореневій директорії репозиторію (проєкт із налаштованими E2E-тестами Playwright) було створено структуру каталогів .github/workflows/ та додано конфігураційний файл ci.yml. У файлі описано автоматичний запуск віртуального середовища Ubuntu, встановлення Node.js, залежностей та браузерів із подальшим запуском E2E-тестів під час подій push та pull_request у гілку main. Вміст файлу наведено у лис. 1. Після додавання файлу та відправлення змін на GitHub, система автоматично ініціювала перевірку. Процес завершився без помилок, усі тести пройдено успішно, що зафіксовано у вкладці Actions (результат наведено на рис. 1).

лис. 1. Конфігураційний файл CI (ci.yml)

```yaml
name: Lab CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ./Lab6

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '20.x'

    - name: Install dependencies
      run: npm install

    - name: Install Playwright Browsers
      run: npx playwright install --with-deps

    - name: Run E2E tests
      run: npx playwright test tests/e2e
```

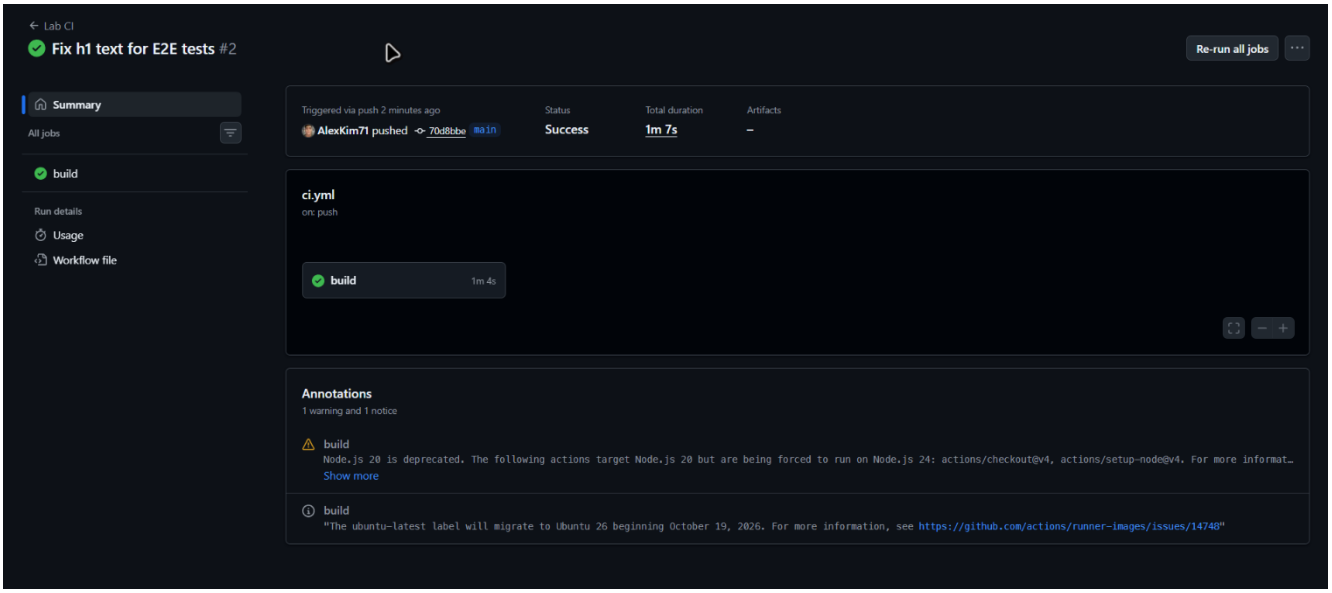


рис. 1. Успішне виконання CI-процесу в гілці main

Завдання 2

Для перевірки реакції системи CI на неробочий код було створено окрему гілку testing-ci-process. У файлах проєкту було навмисно допущено помилку (змінено локатор елемента на сторінці), що призводить до падіння E2E-тесту. Після відправлення цих змін на GitHub було створено Pull Request для злиття тестової гілки з головною гілкою main. Система GitHub Actions автоматично запустила налаштований pipeline, який виявив падіння тестів. У результаті CI-процес завершився зі статусом помилки, а GitHub заблокував можливість злиття неробочого коду (результат перевірки наведено на рис. 2). Це підтверджує ефективність налаштованого Continuous Integration у запобіганні потрапляння дефектів до основної кодової бази

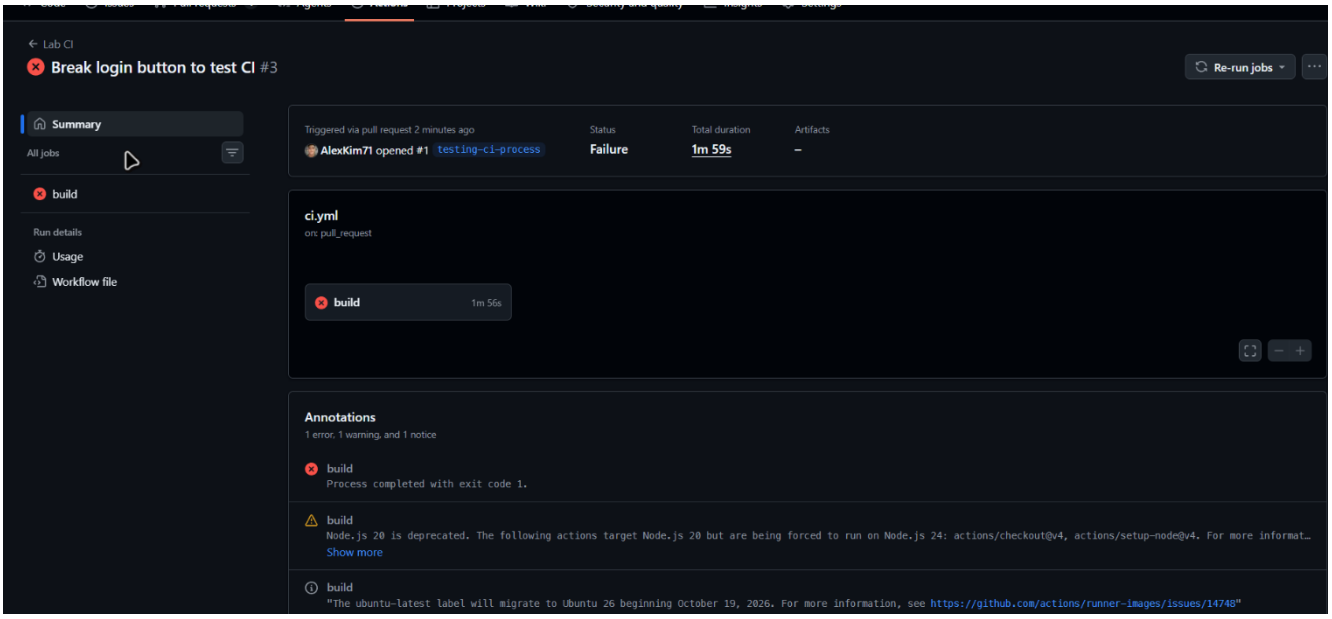


рис. 2. Блокування Pull Request через падіння CI-перевірок

.

Контрольні питання:

1. Що таке Continuous Integration і для чого він використовується? Continuous Integration (CI, безперервна інтеграція) — це практика розробки програмного забезпечення, за якої код від усіх розробників регулярно (кілька разів на день) зливається у спільну головну гілку. Вона використовується для автоматичної збірки проєкту та запуску тестів при кожній зміні, що дозволяє виявляти помилки на ранніх стадіях, уникати "пекла інтеграції" та гарантувати, що новий код не ламає існуючий функціонал.

2. Які інструменти можна використовувати для налаштування CI? Найбільш популярні інструменти на ринку: GitHub Actions (вбудований у GitHub), GitLab CI/CD, Jenkins, CircleCI, Travis CI, Bitbucket Pipelines, Azure DevOps.

3. Як перевірити, чи правильно виконуються CI-процеси у GitHub? Для цього потрібно відкрити вкладку **Actions** у репозиторії проєкту на GitHub. Там відображається історія всіх запусків (workflows). Статус позначається відповідними іконками: зелена галочка (успіх), жовтий кружечок (у процесі) або червоний хрестик (помилка). Натиснувши на конкретний запуск, можна переглянути детальні логи терміналу для кожного кроку (step).

4. Чи можна налаштувати виконання CI-процесів за часом а не за діями? Так, можна. У GitHub Actions для цього використовується тригер schedule. Він налаштовується за допомогою синтаксису cron (наприклад, cron: '0 0 * * *' запустить тестування щодня опівночі), що зручно для важких E2E тестів, які проганяють вночі (Nightly builds).

5. Які переваги автоматичного тестування в процесі CI?

– Швидкий зворотний зв'язок: розробник дізнається про помилку за лічені хвилини після написання коду.

– Блокування дефектів: неможливість влити код (merge Pull Request), якщо тести не пройшли успішно.

– Економія ресурсів: відпадає потреба ручного запуску тестів перед кожним релізом.

– Упевненість у стабільності: головна гілка завжди залишається робочою (deployable).

Висновок:

Під час виконання лабораторної роботи було успішно налаштовано процес Continuous Integration (CI) для вебпроєкту з використанням інструмента GitHub Actions. Створено конфігураційний файл у форматі YAML, який автоматизує підготовку віртуального середовища (встановлення Node.js, браузерів Playwright та залежностей) і запуск E2E тестів при подіях push та pull_request.

На практиці було перевірено роботу CI-пайплайну у двох сценаріях. Позитивний сценарій підтвердив стабільність існуючого коду в головній гілці. Під час перевірки негативного сценарію було створено окрему гілку, куди навмисно додано помилку. Спроба злиття цієї гілки через Pull Request ініціювала автоматичне тестування, яке очікувано завершилося провалом, заблокувавши інтеграцію дефектного коду в main. Мету роботи досягнуто: отримано практичні навички впровадження CI для забезпечення безперервного контролю якості програмного забезпечення.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Репозиторій з вихідним кодом проєкту та налаштованим CI-процесом [Електронний ресурс]. – Режим доступу: https://github.com/AlexKim71/QA-QC

2. Офіційна документація GitHub Actions: створення робочих процесів та налаштування CI [Електронний ресурс]. – Режим доступу: https://docs.github.com/en/actions

3. Офіційна документація фреймворку Playwright для E2E-тестування [Електронний ресурс]. – Режим доступу: https://playwright.dev/

4. Основи Continuous Integration (Безперервної інтеграції) від Atlassian [Електронний ресурс]. – Режим доступу: https://www.atlassian.com/continuous-delivery/continuous-integration

1.